

Agentic AI - Blogger Videoludico Autonomo

Relazione Progetto CCAI A.A. 2025/2026

Riccardo Maria Villaggio

28 giugno 2026

Sommario

Il presente documento illustra la progettazione, lo sviluppo e le soluzioni ingegneristiche adottate per la realizzazione di un sistema di Intelligenza Artificiale Multi-Agente (Agentic AI) specializzato nell'editoria videoludica. Basato sull'infrastruttura architettuale **LangGraph**, il sistema orchestra logiche applicative alimentate da LLM (Llama-3.3-70B), database vettoriali (ChromaDB) e database a grafo (Neo4j) per eseguire in totale autonomia task di pianificazione, ricerca dinamica con Tool Calling, stesura di bozze, validazione Human-in-the-Loop e persistenza semantica. Il sistema integra, inoltre, un modello **BERT Multilingual** sottoposto a fine-tuning (PEFT/LoRA) che agisce come filtro neurale difensivo per il controllo qualità delle fonti web in tempo reale.

Indice

1	Introduzione e Scenario	3
2	Architettura State-Driven e Sistema Multi-Agente	3
2.1	Gestione dello Stato Globale (AgentState)	3
2.2	Il Paradigma Multi-Agente: Giustificazione Architeturale	4
2.3	Struttura del Grafo LangGraph e Casi d'Uso	5
3	Tecnologie, Database e RAG	7
3.1	Knowledge Graph (Neo4j) e Ontologia di Dominio	8
3.2	Architettura K-RAG (Knowledge Graph-Augmented RAG)	8
3.3	Hybrid Search, Reranking e Paginazione (ChromaDB)	9
4	I Tool Ingegnerizzati	9
4.1	Search Tool con Filtro Neurale (search_tool)	10
4.2	RAG Retrieval Tool (rag_retrieval_tool)	10
4.3	Knowledge Graph Tool (knowledge_graph_tool)	10
4.4	Tool di Lettura Profonda (deep_read_article)	11
4.5	YouTube Transcript Fetcher (youtube_transcript_fetcher)	11
5	Dettaglio Fine-Tuning: Filtro Qualità delle Fonti Web	11
5.1	Definizione del Problema nell'Agentic Pipeline	12
5.2	Definizione delle Baseline	12
5.3	Definizione delle Metriche di Valutazione	12

5.4	Definizione e Creazione Automatica del Dataset	13
5.5	Dataset Split	13
5.6	Strategia di Fine-Tuning (PEFT/LoRA)	13
5.7	Model Selection e Ricerca degli Iperparametri (Hyperparameter Search) . .	14
5.8	Confronto Finale con la Baseline	15
5.9	Analisi dei Risultati (Error Analysis e Ipotesi di Miglioramento)	16
5.10	Deployment e Integrazione del Modello (Inferenza)	17
6	Valutazione del Sistema e Osservabilità	17
6.1	Analisi Qualitativa dei Post e Grounding delle Fonti	17
6.2	Identificazione dei Failure Cases e Limiti Architettureali	18
6.3	Parametri di Osservabilità in LangSmith	19
7	Conclusioni e Sviluppi Futuri	20

1 Introduzione e Scenario

Il progetto è nato con l'obiettivo di superare i limiti tipici dei chatbot passivi, strutturando e ingegnerizzando un "Blogger Videoludico Autonomo" capace di gestire logiche proattive analoghe a una vera redazione giornalistica. Il codice sorgente dell'intero ecosistema sviluppato è disponibile sulla repository GitHub ufficiale: https://github.com/RickyV02/CCAI_Project.

La complessità dell'ambiente editoriale — che intrinsecamente richiede un'impeccabile professionalità, l'assenza garantita di allucinazioni fattuali e una pianificazione precisa — ha sancito l'adozione di un **Agentic Workflow**. Grazie al disaccoppiamento logico garantito da **LangGraph**, il sistema ha superato l'esecuzione lineare a favore di una struttura articolata e ricorsiva: un grafo a stati condivisi concepito per interruzioni manuali, implementazione di logiche di problem solving automatizzate e utilizzo dinamico di tool personalizzati.

2 Architettura State-Driven e Sistema Multi-Agente

Al fine di ottemperare al paradigma architetturale del *Single Responsibility Principle* (Principio di Singola Responsabilità), l'entità monolitica del LLM è stata scissa in Nodi funzionali dedicati.

2.1 Gestione dello Stato Globale (AgentState)

Tutti i vertici del grafo operano su uno stato applicativo globalmente condiviso (**AgentState**), il quale funge da memoria transazionale del sistema. Questo dizionario tipizzato traccia in tempo reale le seguenti variabili critiche:

- **user_input**: L'input testuale iniziale dell'utente (un gioco specifico o una richiesta di suggerimento).
- **reasoning_trace**: La traccia strutturata delle operazioni (Thought, Action, Observation) per mantenere interpretabile il processo decisionale ReAct.
- **tool_outputs**: Gli output grezzi generati dai tool invocati.
- **kg_context**: I dati di dominio estratti dal Knowledge Graph.
- **planning_information**: Contiene tutte le informazioni relative al piano editoriale generato.
- **research_summary**: L'estrazione fattuale strutturata post-ricerca, epurata dal rumore web.
- **draft_post**: La bozza temporanea (e poi finale) dell'articolo.
- **human_feedback**: Le istruzioni o critiche fornite dall'utente in fase di revisione.
- **quality_passed**: Un flag booleano che attesta il superamento del Quality Check automatico.
- **revision_count**: Un contatore delle revisioni automatiche per prevenire loop infiniti in fase di stesura.

2.2 Il Paradigma Multi-Agente: Giustificazione Architettureale

La definizione di "Sistema Multi-Agente" in questa architettura non è dettata dalla mera distribuzione di tool esterni a tutti i nodi, ma dal **ruolo funzionale e cognitivo** che ogni vertice svolge. Sebbene il sistema sia alimentato da un unico LLM di base (Llama-3.3-70B), ogni nodo lo invoca dotandolo di un *System Prompt* rigorosamente separato e di specifiche responsabilità, coordinate da logiche Python deterministiche. Un agente è tale poiché persegue un proprio obiettivo, elabora lo stato secondo direttive esclusive e genera output che altri agenti consumeranno.

Limitare l'arsenale dei tool esclusivamente al nodo *Researcher* rappresenta una precisa e ponderata **scelta ingegneristica**:

- **Accuratezza e Risparmio Token:** Assegnare i tool solo al ricercatore massimizza la precisione delle chiamate, in quanto sono stati pensati appositamente per il task che questo nodo deve svolgere. In questo modo concentriamo il contesto unicamente su di lui, responsabile in prima persona della raccolta dati, evitando di disperdere token in tool inutili per gli altri agenti, o di dover separare il contesto in più prompt per più nodi. Inoltre, i tool sono stati progettati appositamente per essere usati dalla stessa entità cognitiva, evitando di doverli adattare a contesti diversi e riducendo il rischio di errori di interpretazione.
- **Separazione dei Contesti Cognitivi:** Il *Researcher* necessita di una logica esplorativa, flessibile e tollerante agli errori web; il *Writer* richiede un rigore puramente editoriale e testuale; il *Memory Updater* deve operare come un connettore al database transazionale, limitandosi a fare *Named Entity Recognition* (NER) nativa tramite output strutturati, rendendo di fatto inutile fornirgli un tool esplicito per l'aggiornamento. Unificare queste direttive conflittuali degraderebbe la capacità del modello di seguire le istruzioni.
- **Iniezione del pattern Human-in-the-Loop:** Dal punto di vista dell'ingegneria del software, la scomposizione in nodi discreti in LangGraph è l'unico modo sicuro per congelare lo stato dell'applicazione e cedere il controllo all'operatore umano. Dividendo logicamente i ruoli in agenti separati, è possibile intercettare il flusso dopo la stesura ma rigorosamente prima dell'aggiornamento persistente (Neo4j).

2.3 Struttura del Grafo LangGraph e Casi d'Uso

La topologia completa del sistema è illustrata nella Figura 1.

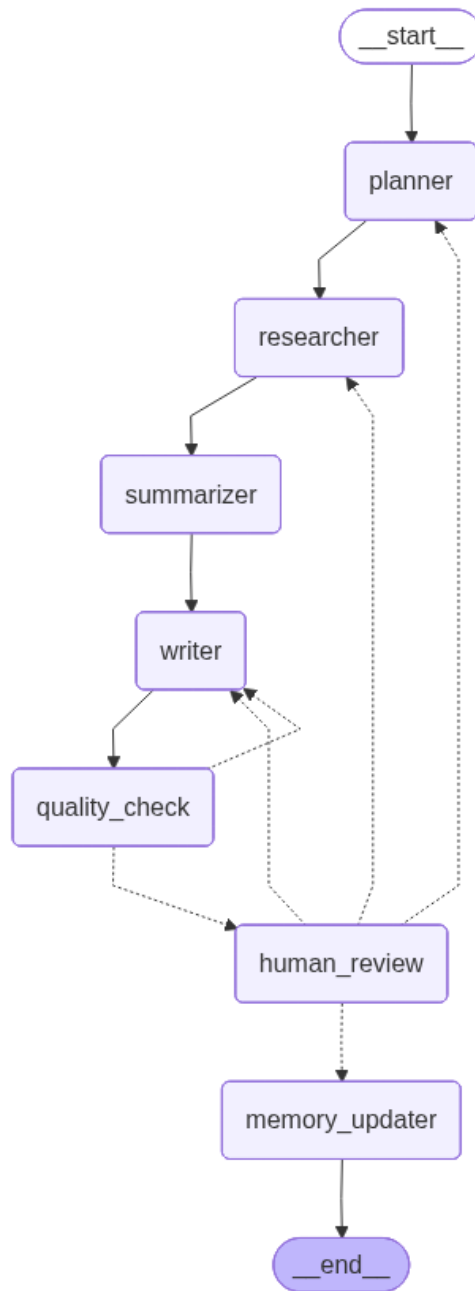


Figura 1: Grafo dei flussi e nodi decisionali del sistema multi-agente in LangGraph.

Il flusso di esecuzione si articola nei seguenti nodi interconnessi, progettati per gestire in modo dinamico i diversi casi d'uso richiesti dal dominio applicativo:

1. **Planner Node (Direzione Editoriale):** Si occupa della definizione del tema e del piano editoriale. L'ingegnerizzazione di questo nodo sfrutta un modello Pydantic estremamente strutturato (`SuggestPlannerOutput`) che obbliga l'LLM a non allucinare. Il modello richiede la compilazione di uno *scratchpad privato* in cui l'agente deve verificare obbligatoriamente ogni singolo titolo rispetto a una blacklist e

al catalogo di Neo4j. In questo modo si garantisce matematicamente che l'agente proponga solo giochi idonei. Il nodo può operare in due modalità distinte:

- *Modalità Autonoma*: Se lasciato agire liberamente, suggerisce un calendario editoriale di tre videogiochi completamente nuovi. Interroga il Knowledge Graph (KG) per assicurarsi in primis che siano titoli mai recensiti all'interno del blog; viceversa, se il KG è vuoto o non soddisfa la richiesta, proporrà titoli nuovi tratti dalla sua conoscenza generale. L'utente può approvare, modificare, rifiutare le proposte o fornire direttive precise (es. "Voglio un piano editoriale di soli giochi horror del 2022"). Infine, il nodo ricorda anche l'eventuale presenza di un piano editoriale approvato in precedenza e in corso di pubblicazione settimanale, dunque, se l'utente desidera, si procederà alla stesura della recensione del prossimo gioco in coda.
- *Modalità su Richiesta*: L'utente può specificare un gioco preciso. Qualora l'utente richieda un gioco *già recensito* nel blog, il sistema avviserà l'utente ed eviterà di ripetere lo stesso contenuto, proponendo la stesura di un articolo con un **angolo specifico e inedito** (es. "Focus esclusivo sulla lore" o "Analisi dei boss"). Tale angolo può essere scelto dall'utente o dedotto automaticamente dall'agente analizzando le recensioni pregresse.

2. **Researcher Node (Investigatore Data-Driven)**: Opera in due fasi. Una prima fase deterministica (ricerca web base e interrogazione entità KG) e una seconda fase *Agentica* basata su un ciclo **ReAct** (Thought → Action → Observation). L'agente sfrutta vari strumenti:

- `knowledge_graph_tool`: interroga il database a grafi per estrarre informazioni note. Rappresenta la primissima operazione eseguita per inquadrare il contesto iniziale e generare query mirate.
- `search_tool`: esegue ricerche web filtrate da un modello BERT-based per scaricare siti e-commerce o non giornalistici. Risulta fondamentale per popolare in real-time il database vettoriale prima dell'analisi.
- `rag_retrieval_tool`: esegue query semantiche (Hybrid Search) direttamente sui documenti testuali e sugli articoli precedentemente scaricati e inseriti in ChromaDB, permettendo all'agente di interrogare a fondo il materiale raccolto.
- `youtube_transcript_fetcher`: estrae istantaneamente le trascrizioni testuali dai video per ottenere recensioni o saggi complessi.
- `deep_read_article`: **Soluzione ingegneristica chiave** ideata per superare il limite massimo di token in input al modello utilizzato (vincolo critico nell'utilizzo di LLM via API Groq). Questo tool permette all'agente di leggere articoli molto lunghi "a blocchi" scorrendo il testo utilizzando parametri dinamici di *offset* e *limit*.

3. **Summarizer Node (Analista Dati)**: Raccoglie tutti gli output generati dal Researcher. Scarta le fonti irrilevanti o fuori contesto (es. segnalazioni di spazzatura o devianze da franchise simili) e compatta le informazioni in un template strutturato. Fornisce così al *Writer* una base di dati verificata e priva di allucinazioni o distrazioni. In particolare, compila due schemi Pydantic rigidi:

- **SourceEvaluation**: per ogni sito utilizzato, valuta il nome della testata, l'URL, la pertinenza e la credibilità.
 - **GameResearchExtraction**: obbliga il modello a strutturare le informazioni in categorie chiare (es. lore, meccaniche, boss, dettagli tecnici, valutazioni). Contiene inoltre un campo vitale di **fact_check_notes** in cui l'agente è obbligato a giustificare perché ha scartato o ritenuto non pertinenti determinate fonti (es. listini prezzi).
4. **Writer Node (Il Redattore)**: Armato di vincolanti direttive (*System Prompt*), utilizza i dati puliti del Summarizer per redigere la recensione imponendo un tono tipico da blog. In caso di fallimento del Quality Check o in presenza di critiche umane, il nodo si riattiva per rielaborare la stesura e applicare le modifiche richieste.
 5. **Quality Check Node (Revisore Automatico)**: Sfrutta un LLM configurato come giudice logico. Valuta se la bozza rispetta metriche precise (es. cita le fonti? Parla dei boss in modo coerente? La lunghezza è adeguata?). Se il verdetto è negativo, istruisce il Writer sulle modifiche da apportare, garantendo l'auto-correzione per un numero massimo predefinito di cicli.
 6. **Human Review Node (Controllo Umano)**: Interrompe l'autonomia del sistema per cedere il controllo all'utente, fungendo da snodo di *Routing* per quattro scenari:
 - *Approve*: L'articolo è perfetto, il flusso procede all'aggiornamento dei database.
 - *Rewrite*: L'utente richiede modifiche stilistiche o di lunghezza (es. "scrivila più corta", "traduci in inglese") mantenendo intatto l'argomento. Il flusso regredisce al Writer.
 - *Need Research*: L'utente esige maggiore profondità (es. "parla più nel dettaglio dei combattimenti"). Il flusso torna in modo ricorsivo al Researcher per raccogliere nuovi dati.
 - *Change Topic*: L'utente decide di scartare il lavoro e cambiare tema. Il sistema effettua un hard-reset dello stato e torna al Planner.
 7. **Memory Updater Node (Storicizzatore)**: Agisce come modulo NER (*Named Entity Recognition*). Sfruttando il modello Pydantic **PostEntities**, il nodo estrae semantica dall'articolo approvato (generi, studios, boss citati, piattaforme). Un dettaglio progettuale rilevante riguarda le **Fonti**: si è scelto di estrarre e memorizzare nel Knowledge Graph il *nome della testata* (es. "IGN Italia") anziché l'URL univoco della pagina. Questo permette a Neo4j di raggruppare i nodi efficacemente (es. visualizzare tutti i giochi recensiti citando "IGN Italia"), evitando un'inutile frammentazione del grafo causata da link singoli. Oltre all'aggiornamento in Neo4j, il nodo archivia il testo integrale dell'articolo in ChromaDB.

3 Tecnologie, Database e RAG

Il motore di ragionamento principale dell'architettura è basato sul modello **Llama-3.3-70B** (servito via API Groq), accuratamente selezionato per le altissime prestazioni nel *Tool Calling* nativo e per la latenza inferenziale quasi nulla, caratteristica imprescindibile in un sistema in cui il loop ReAct esige dozzine di interazioni sequenziali.

A supporto del modello, la persistenza e il recupero della conoscenza sono demandati a un'architettura duale formata da un database a grafi (Neo4j) e un database vettoriale (ChromaDB), integrati tramite il paradigma K-RAG.

3.1 Knowledge Graph (Neo4j) e Ontologia di Dominio

Per superare la volatilità della *Context Window* dell'LLM e garantire una memoria di lungo termine strutturata, il sistema impiega un database a grafi nativo (**Neo4j**). Il modulo **KGManager** interagisce con il database in modo asincrono, sfruttando il recente standard **Model Context Protocol (MCP)**. Questa scelta ingegneristica isola l'ambiente di esecuzione di Neo4j e previene il blocco dell'event loop principale (Event Loop Blocking) durante l'elaborazione parallela dei nodi LangGraph.

L'ontologia del Knowledge Graph è rigidamente predefinita da un file di inizializzazione (`init.kg.cypher`) che impone vincoli di unicità (*Constraints*) sulle entità per evitare duplicazioni semantiche. La struttura dei dati si articola sui seguenti rami:

- **Entità di Dominio (Nodi):** Game, Boss, Mechanic, Genre, Studio, Platform, Character. A queste si aggiungono le entità editoriali: BlogPost, EditorialPlan, Source e Claim.
- **Relazioni (Edge):** I nodi sono interconnessi da archi direzionali altamente descrittivi. Ad esempio, un videogioco è strutturato tramite le relazioni `[ :HAS_BOSS ]`, `[ :USES_MECHANIC ]`, `[ :PART_OF_GENRE ]`, `[ :AVAILABLE_ON ]` e `[ :SIMILAR_TO ]`. Dal lato editoriale, la relazione `[ :COVERED_IN ]` collega il gioco all'articolo pubblicato, che a sua volta traccia le fonti tramite `[ :USED_SOURCE ]`.

L'utilizzo attivo del KG pervade l'intera pipeline:

1. **Fase di Planning:** Il Planner interroga il KG (`query_all_games` e `check_existing_reviews`) per mappare lo storico del blog. Questo previene in modo assoluto la ridondanza: se un titolo ha già un arco `[ :COVERED_IN ]`, il Planner è costretto a proporre un videogioco diverso o a cambiare l'angolo della recensione.
2. **Deducibilità delle Somiglianze:** Tramite la funzione `get_catalog_for_similarity`, il sistema estrae un'impronta digitale dei giochi (incrociando sviluppatori, meccaniche e generi). L'LLM utilizza queste tracce per dedurre e creare autonomamente nuovi archi `[ :SIMILAR_TO ]` al termine di una recensione.
3. **Aggiornamento Transazionale (NER):** Dopo l'approvazione umana, il *Memory Updater* compila lo schema Pydantic `PostEntities` per svolgere una vera e propria estrazione di relazioni (Relation Extraction). Il **KGManager** formatta dinamicamente costrutti Cypher tramite cicli `FOREACH` e comandi `MERGE`, consolidando le nuove nozioni in modo atomico.

3.2 Architettura K-RAG (Knowledge Graph-Augmented RAG)

Il sistema risolve le tipiche inefficienze dei sistemi RAG tradizionali — spesso vittime di ambiguità terminologica (Lost in the Middle) — implementando un approccio ibrido, delegato al modulo **RAGManager**.

La pipeline di recupero implementa la strategia **Knowledge Graph-Augmented RAG (K-RAG)**. Prima di ricercare informazioni testuali alla cieca, il *Researcher* pre-interroga Neo4j per estrarre le entità note sul gioco investigato. Il codice impiega queste stringhe di contesto per "fabbricare" un set di query espanse (*Multi-Query Expansion*). Ad esempio, invece di cercare genericamente il nome del gioco, il codice genera iterazioni come: "[Topic] boss difficoltà" o "[Topic] meccaniche gameplay".

3.3 Hybrid Search, Reranking e Paginazione (ChromaDB)

I documenti grezzi acquisiti dalla ricerca web e dallo scraping delle trascrizioni YouTube vengono iniettati localmente nel database vettoriale **ChromaDB** con una frammentazione strategica (`chunk_size=1000`, `overlap=100`). La fase di recupero (*Retrieval*) opera in due stadi ad alta precisione:

1. **Stage 1 - Retrieval Ibrido (Dense + Lexical Search):** Il sistema non si affida solo al calcolo dei coseni vettoriali (Dense Retrieval operato tramite `HuggingFaceEmbeddings` con il modello `all-MiniLM-L6-v2`), che eccelle nella comprensione semantica ma fallisce sulle parole chiave specifiche. L'istanza ChromaDB è stata affiancata all'algoritmo lessicale **BM25Okapi**. I risultati dei due motori (candidati densi + candidati lessicali basati su frequenza esatta dei termini) vengono recuperati parallelamente e unificati.
2. **Stage 2 - Cross-Encoder Reranking:** L'unione dei candidati produce una lista ricca ma non ordinata. Per ottimizzare la selezione, il sistema impiega un modello **Cross-Encoder** (`ms-marco-MiniLM-L-6-v2`). A differenza dei bi-encoder, che misurano la distanza tra due vettori indipendenti, il cross-encoder processa contemporaneamente la coppia `[[Query], [Documento]]` attraverso l'architettura Transformer, restituendo un *relevance score* probabilistico estremamente più accurato. Solo la top-K dei risultati riordinati viene effettivamente inviata all'LLM.
3. **Deep Read Pagination (Offset Search):** Per eludere in modo ingegneristico i limiti massimi dei token di input dell'LLM (senza perdere informazioni preziose di lunghi articoli o saggi di YouTube), il **RAGManager** implementa il metodo `get_article_chunks`. Sfruttando i metadati registrati in fase di ingestione (`chunk_index` e `source_url`), l'agente può invocare un tool per scorrere letteralmente la pagina web, caricando a blocchi consecutivi i paragrafi di interesse tramite la manipolazione dei campi *limit* e *offset*.

4 I Tool Ingegnerizzati

Tutti gli strumenti software a disposizione della pipeline agentica sono stati interamente ingegnerizzati, decorati tramite il costrutto `@tool` di LangChain e centralizzati all'interno dello script sorgente `tools.py`. In conformità con i requisiti di *Reasoning and Tool Usage*, ogni risorsa abilita la selezione dinamica da parte del nodo *Researcher*, supportata da una chiara giustificazione d'uso documentata nella traccia di ragionamento (`reasoning_trace`) a ogni ciclo di esecuzione ReAct.

4.1 Search Tool con Filtro Neurale (`search_tool`)

Rappresenta il principale strumento per l'esplorazione e l'acquisizione di informazioni esterne.

- **Raccolta e Scraping Ibrido:** Utilizza l'API di Tavily per il fetching dei risultati, ma non si limita ai riassunti pre-generati dai motori di ricerca. Il codice scarica il DOM completo (raw HTML) ed estrae il testo utile tramite la libreria `trafilatura`, implementando un fallback automatico su `BeautifulSoup` in caso di fallimento o di blocchi anti-bot. Applica inoltre una sanitizzazione tramite espressioni regolari (Regex) per rimuovere URL spazzatura e artefatti markdown.
- **Filtro AI Difensivo (Defensive RAG):** Prima di salvare i documenti, il tool intercetta il testo e invoca un'istanza Singleton del modello BERT-based sottoposto a fine-tuning (`MLEvaluator`). Ogni pagina viene analizzata semanticamente: se classificata come "Spazzatura/Store" (es. e-commerce o clickbait), la fonte viene scartata a monte.
- **Giustificazione Architetture (Risparmio Token):** Si è scelto deliberatamente di inglobare il modello AI all'interno del tool di ricerca, **invece di incorporarlo come tool autonomo** a disposizione dell'agente. Poiché lo scraping e il filtraggio sono operazioni logicamente sequenziali, costringere l'LLM a scaricare il raw text, leggerlo (consumando migliaia di token di contesto), decidere di invocare un secondo tool di "pulizia" e attenderne l'output, avrebbe generato una latenza inaccettabile e uno spreco massivo di risorse. Inglobandolo, il filtro agisce in background a costo zero.
- **Ingestion Dinamica:** Il testo validato dal modello viene frammentato (*chunking*) e salvato istantaneamente nel database vettoriale ChromaDB, rendendolo da subito interrogabile.

4.2 RAG Retrieval Tool (`rag_retrieval_tool`)

Costituisce l'interfaccia di comunicazione diretta tra l'agente e il database vettoriale locale.

- **Interrogazione Semantica:** Permette all'LLM di formulare domande discorsive (es. "Come funziona il combat system di questo gioco?") per recuperare i frammenti informativi più pertinenti dai documenti scaricati durante la sessione o archiviati storicamente.
- **Routing verso l'Hybrid Search:** Il tool innesca in background le logiche del `rag_manager`, attivando la pipeline di recupero ibrido (Dense Embeddings + BM25) e il successivo Cross-Encoder Reranking, assicurando che l'agente riceva sempre il set di informazioni a più alta densità semantica.

4.3 Knowledge Graph Tool (`knowledge_graph_tool`)

Strumento di esplorazione ontologica dedicato alla lettura del database a grafi (Neo4j).

- **Estrazione Relazionale:** Interroga Neo4j per estrarre in tempo reale tutti i meta-dati e i nodi noti correlati a un videogioco specifico (generi, sviluppatori, meccaniche, recensioni passate).

- **Ancoraggio al Contesto:** Costituisce la primissima *tool call* obbligatoria dettata nel prompt del *Researcher*. Fornisce un’istantanea fattuale che l’agente utilizza per pianificare e ottimizzare le keyword delle successive ricerche web, mitigando il rischio di allucinazioni esplorative.

4.4 Tool di Lettura Profonda (*deep_read_article*)

Rappresenta il primo dei due tool **custom** progettati, sviluppato come soluzione ingegneristica per superare le limitazioni hardware dei modelli linguistici.

- **Bypass della Context Window:** L’iniezione diretta di un articolo web da 10000 parole saturerebbe rapidamente il limite di token dell’LLM. Questo tool fornisce all’agente la capacità di ”paginare” la lettura.
- **Navigazione tramite Offset:** L’agente invoca lo strumento specificando un URL target, un *offset* di partenza e un *limit*. Il sistema interroga ChromaDB filtrando esattamente per il metadato `source_url` e per l’indice del chunk, restituendo un blocco limitato di paragrafi. L’agente può iterativamente aumentare l’offset per proseguire la lettura solo qualora la fonte si stia rivelando pertinente ai fini della ricerca.

4.5 YouTube Transcript Fetcher (*youtube_transcript_fetcher*)

Il secondo tool **custom** ingegnerizzato, strategicamente vitale nel dominio videoludico dove saggi visivi e video-recensioni rappresentano fonti di altissimo valore editoriale.

- **Integrazione API e Parsing:** Ricevuto un URL di YouTube (intercettato dalla web search), il tool estrae dinamicamente l’ID del video ed esegue il download dell’intera trascrizione tramite `youtube_transcript_api`, gestendo nativamente il multilinguismo (italiano e inglese).
- **Vettorializzazione on-the-fly:** Per evitare saturazioni della context window, la trascrizione grezza non viene stampata nell’output dell’agente. Al contrario, viene automaticamente frammentata, arricchita con i relativi metadati strutturati (topic, URL, source type) e iniettata silenziosamente in ChromaDB.
- **Sinergia Inter-Tool:** Il tool restituisce all’agente un log di successo con il conteggio delle parole (es. ”Trascrizione salvata: 40 chunk”). Questa architettura spinge l’agente a concatenare l’uso degli strumenti: dopo aver vettorializzato il video, l’LLM invocherà iterativamente `deep_read_article` o `rag_retrieval_tool` sull’URL appena indicizzato per estrarne analiticamente i contenuti.

5 Dettaglio Fine-Tuning: Filtro Qualità delle Fonti Web

In conformità con le specifiche di progetto, è stato addestrato e integrato un modello di Intelligenza Artificiale personalizzato. Di seguito vengono analizzati i singoli step metodologici che hanno formalizzato il processo.

5.1 Definizione del Problema nell'Agentic Pipeline

Il componente della pipeline agentica oggetto di ottimizzazione è il modulo di *Information Retrieval* associato al `search_tool`. Durante le fasi di testing, l'agente *Researcher* subiva un grave inquinamento del contesto: interrogando i motori di ricerca con i titoli dei videogiochi, le API restituiscono occasionalmente siti di E-commerce (Amazon, Instant Gaming, Steam, eBay) oppure pagine di forum non affidabili, invece di vere recensioni giornalistiche. L'obiettivo è creare un classificatore binario capace di distinguere:

- **Classe 1 (Informative):** Articoli, recensioni e wiki testualmente ricchi.
- **Classe 0 (Junk/Store):** Negozi digitali, listini prezzi, forum generici e pagine povere di contenuto utile.

L'output del modello viene consumato in modo deterministico dal sistema: i documenti classificati come *Junk (0)* vengono eliminati a monte, evitando l'inserimento nel database vettoriale (RAG).

5.2 Definizione delle Baseline

Al fine di valutare quantitativamente l'efficacia del processo di fine-tuning, sono state definite due distinte baseline di confronto:

1. **Baseline 1 (Heuristic Rule):** Un algoritmo deterministico Python (Regex/Keyword Matching) che assegna la Classe 0 se il testo contiene un pattern tipico delle pagine spazzatura di nostro interesse ("acquista", "carrello", "prezzo", "spedizione", "offerta", "login") o se la lunghezza della pagina è inferiore a 30 caratteri. Serve a quantificare quanto il task sia risolvibile tramite programmazione standard.
2. **Baseline 2 (Zero-Shot AI):** Utilizzo del modello pre-addestrato generico MoritzLaurer/mDeBERTa-v3-base-mnli-xnli in modalità *Zero-Shot Classification*. Misura se un'architettura NLI (Natural Language Inference) avanzata riesca a risolvere il task senza alcun adattamento di dominio.

5.3 Definizione delle Metriche di Valutazione

Considerata la natura del task (classificazione binaria) e la varianza strutturale del dominio, il modello è stato valutato impiegando diverse metriche a seconda della fase operativa:

- **Fase di Validation (Model Selection):** Durante la ricerca degli iperparametri, sono state utilizzate **Accuracy** e **Macro F1-Score** per monitorare la convergenza. Il Macro F1-Score ha rappresentato la metrica decisionale primaria poiché, calcolando la media armonica non pesata, assicura che il modello impari a distinguere equamente sia le fonti utili che quelle spazzatura.
- **Fase di Test Finale (Error Analysis):** In fase di scrutinio sul Test Set, l'analisi è stata estesa estrapolando dal Classification Report anche le metriche di **Precision** e **Recall** per singola classe. Questo permette di mappare esattamente il comportamento del classificatore, isolando l'impatto dei Falsi Positivi rispetto ai Falsi Negativi.

5.4 Definizione e Creazione Automatica del Dataset

Non esistendo in letteratura un dataset pubblico specifico per il filtraggio di fonti videoludiche, il corpus dati (`source_quality_dataset.csv`) è stato interamente ingegnerizzato ad hoc sviluppando uno script dedicato (`create_dataset.py`). Per garantire che il modello venisse addestrato sull'esatta distribuzione di dati che incontrerà in produzione, la pipeline di acquisizione ha automatizzato i seguenti passaggi:

1. **Scraping Reale:** Lo script ha interrogato l'API di Tavily simulando dozzine di query utente, estraendo dal raw HTML i titoli e gli snippet delle pagine. Tramite euristica, i testi troppo brevi (sotto i 30 caratteri) sono stati scartati.
2. **LLM-Assisted Labeling:** Per l'annotazione non si è proceduto unicamente a mano. Gli snippet sono stati dati in pasto a un LLM veloce (`llama-3.1-8b-instant` via Groq), vincolandone rigidamente la risposta tramite la libreria *Pydantic* (`LabelerOutput`) affinché restituisse unicamente un intero: **1 (Articolo Utile)** o **0 (E-commerce/Junk/Forum)**.

Questa procedura semi-supervisionata, validata in un secondo momento con controllo a campione, ha prodotto un dataset di 265 istanze altamente rappresentativo della reale rumorosità del Web. Di seguito è riportato il formato di un campione di input del train set:

```
LABEL: Articolo/Utile (1)
TESTO: LA RECENSIONE PIU' ONESTA CHE TROVERAI DI ZELDA TOTK NO SPOILERS
Frankie's Lair 31200 subscribers. 80 ore di gioco certificate dalle mie Live.
Ho finito Zelda Tears of the Kingdom col True Ending ed oggi sono qui a
portarvi la recensione più onesta che potrete mai trovare...
```

5.5 Dataset Split

Per garantire robustezza e scongiurare fenomeni di *Data Leakage*, i 265 campioni sono stati suddivisi utilizzando un Random Seed pari a 42:

- **Training Set:** 185 campioni (69.8%). Distribuzione: 108 esempi di Classe 1 e 77 di Classe 0.
- **Validation Set:** 40 campioni (15.1%), utilizzati per la selezione del modello e degli iperparametri.
- **Test Set:** 40 campioni (15.1%), mantenuti completamente separati dal processo di addestramento e utilizzati esclusivamente per la valutazione finale.

5.6 Strategia di Fine-Tuning (PEFT/LoRA)

Il Foundation Model selezionato per il task è **bert-base-multilingual-cased**. La versione "cased" preserva le maiuscole, fondamentali nel dominio gaming, mentre la natura multilingua copre nativamente testate italiane ed estere.

Per l'addestramento, si è deliberatamente evitato di ricorrere a un *Full Fine-Tuning* dell'intera architettura. Questa decisione è stata presa riflettendo sulla natura stessa del

task: la *Sequence Classification* è un task NLP standard per il quale i modelli encoder (come BERT) possiedono già un'eccellente rappresentazione latente della lingua. Pertanto, l'adozione di tecniche di **Parameter-Efficient Fine-Tuning (PEFT)** tramite l'algoritmo **LoRA** (Low-Rank Adaptation) è largamente sufficiente a eguagliare le performance di un full tuning, ma con una frazione del costo computazionale.

L'architettura di addestramento è stata così modulata:

- Il corpo pre-addestrato di BERT è stato **congelato** (frozen pesi originali).
- Sono state iniettate e addestrate le matrici di LoRA unicamente nei layer di attenzione (*query*, *value*).
- **La testa di classificazione (Classifier Head)** sequenziale finale è stata sbloccata e addestrata ex novo per mappare i parametri sulle due classi target.

A differenza di alcuni tentativi presenti in letteratura, non è stato applicato alcun troncamento forzato in pre-processing a 128 token. Sfruttando interamente la finestra architetturale di BERT (`truncation=True`, limite nativo di 512 token), è stato permesso all'algoritmo di attingere alle porzioni centrali delle pagine web, dove tipicamente risiedono le *call to action* fraudolente ("Aggiungi al Carrello", "Policy Resi"). Per casi in cui i testi sono brevi ed è facilmente intuibile la natura, troncamento non è rischioso, anzi diminuisce la latenza e i costi; tuttavia, nel nostro caso, si è dimostrato estremamente vantaggioso lasciare il modello libero di vedere più testo possibile, di fatti le metriche hanno avuto notevoli miglioramenti (2 errori in meno sul Test Set).

5.7 Model Selection e Ricerca degli Iperparametri (Hyperparameter Search)

Al fine di identificare la configurazione ottima senza indurre overfitting su un dataset dimensionalmente ridotto, è stata eseguita una *Grid Search* variando il Learning Rate (`lr`) e il Rango di espansione LoRA (`r`).

È fondamentale sottolineare una scelta metodologica: parametri di regolarizzazione come il **Dropout** (0.1) e il **Weight Decay** (0.01) sono stati mantenuti volutamente fissi. L'obiettivo era concentrarsi sull'impatto esplorativo del rango e del learning rate per verificare se la loro sola ottimizzazione fosse sufficiente alla risoluzione del task. Il parametro `lora_alpha` è stato mantenuto fisso a 32, introducendo una variazione implicita dello scaling `alpha/r` tra le configurazioni (da 16 a 0.5). Nonostante questo confounding metodologico, la saturazione delle prestazioni osservata da `r=8` in poi — con F1 stabile a 0.9246 su scaling decrescente — indica che la selezione del modello ottimo non è stata influenzata da tale variazione.

Il problema del Determinismo e della Precisione Mista: Per garantire la riproducibilità della Grid Search e mitigare la sensibilità del modello, è stato forzato il determinismo hardware introducendo le seguenti direttive:

```
torch.cuda.manual_seed_all(SEED)
torch.backends.cudnn.deterministic = True
torch.backends.cudnn.benchmark = False
```

Senza queste configurazioni, le operazioni CUDA restano non-deterministiche tra una run e l'altra, poiché alcune computazioni sulle GPU (es. somme parallele, operazioni di attenzione) possono restituire output marginalmente diversi a causa di come i thread vengono

schedulati sull’hardware. Questo fenomeno è drammaticamente amplificato dall’uso della precisione mista (`fp16=True`): i numeri in virgola mobile a 16 bit possiedono una risoluzione molto più bassa. Piccole variazioni numeriche durante il training si propagano e possono spostare i pesi del modello di quel tanto che basta a classificare diversamente uno o due esempi borderline. Di conseguenza, la *Hyperparameter Search* risulta intrinsecamente suscettibile a piccole variazioni: se due configurazioni ottengono un Validation F1 molto vicino, una minima variazione numerica può far ”vincere” un rango diverso (es. `r=8` piuttosto che `r=16` o `r=32`) rispetto a un’altra run. Tuttavia, come precisato e come riscontrabile dai risultati, la saturazione prestazionale a partire da `r=8` indica che il modello ottimale è stato correttamente identificato, ed inoltre, tra le diverse configurazioni migliori durante le eventuali oscillazioni, i risultati ottenuti in termini di metriche convergono sempre a valori estremamente simili, garantendo la robustezza del modello finale.

Le ipotesi sperimentali finali, testate con i vincoli deterministici attivati (fissate a 5 epoche e `alpha=32`), hanno prodotto sul **Validation Set** (40 campioni) i risultati delineati nella tabella seguente:

Configurazione (Ipotesi Sperimentale)	Iperparametri	Val F1-Score	Val Accuracy
1: Baseline LR Conservativo (Sotto-ottimizzato)	lr: 5e-5, r: 8	0.4643	0.5500
2: LR Aggressivo, Basso Rango	lr: 2e-4, r: 2	0.8743	0.8750
3: LR Aggressivo, Rango Medio	lr: 2e-4, r: 4	0.8990	0.9000
4: LR Aggressivo, Rango Ottimo (Parametri: 296K)	lr: 2e-4, r: 8	0.9246	0.9250
5: LR Aggressivo, Espansione Larga	lr: 2e-4, r: 16	0.9246	0.9250
6: LR Aggressivo, Espansione Estesa	lr: 2e-4, r: 32	0.9246	0.9250
7: LR Aggressivo, Espansione Massima	lr: 2e-4, r: 64	0.9246	0.9250

Tabella 1: Risultati della Grid Search valutati sul Validation Set.

Analisi delle configurazioni: La Configurazione 1, a causa di un learning rate troppo basso (5e-5), ha palesemente fallito la convergenza, equiparandosi quasi a un lancio di moneta (Acc: 0.55). Alzando il learning rate a 2e-4 la rete ha iniziato ad apprendere correttamente. Il *plateau* prestazionale è stato raggiunto dalla **Configurazione 4 (r=8)**, che ha massimizzato le metriche (F1: 0.9246). Aumentare ulteriormente il rango (Configurazioni 5, 6 e 7) non ha prodotto alcun miglioramento, stabilizzando i risultati in una fase di saturazione. Pertanto, l’algoritmo ha isolato e serializzato la Configurazione 4, massimizzando l’efficienza parametrica rispetto ai modelli più pesanti.

5.8 Confronto Finale con la Baseline

I pesi del modello vincitore (LoRA Config 4) sono stati caricati ed eseguiti contro le due Baseline utilizzando esclusivamente il **Test Set** (40 campioni) per misurare il grado reale di generalizzazione.

Livello e Approccio	Test Accuracy	Test Macro F1
Livello 1 - Baseline Deterministica (Heuristic Rule)	0.7000	0.6581
Livello 2 - Baseline Zero-Shot AI (mDeBERTa-v3)	0.8750	0.8711
Livello 3 - Modello Fine-Tuned (BERT+LoRA)	1.0000	1.0000

Tabella 2: Comparazione finale sul Test Set (40 campioni).

Il modello Fine-Tuned ha superato nettamente i riferimenti di base, registrando in questa esecuzione un **Miglioramento Assoluto del +30.00%** rispetto all'euristica standard e del +12.50% sull'AI Generalista, provando empiricamente il valore di un addestramento specializzato (Domain Adaptation).

5.9 Analisi dei Risultati (Error Analysis e Ipotesi di Miglioramento)

Le metriche ottenute nell'esecuzione ottimizzata dimostrano una solidità assoluta del classificatore (Accuracy 100.0%, 0 errori). È tuttavia essenziale operare una precisazione oggettiva: a causa del supporto ridotto del Test Set (40 campioni) e della sensibilità in virgola mobile residua dell'architettura hardware spiegata in precedenza, in diverse iterazioni empiriche l'accuracy del modello si attesta in un range fisiologico **tra il 95% e il 100%** (commettendo al massimo da 1 a 2 errori totali).

Quando il modello fallisce in queste run minoritarie, le disallineazioni si concentrano su esempi semanticamente ambigui (*borderline*), la cui classificazione è instabile. L'analisi storica di questi *Failure Cases* tipici ha evidenziato due scenari strutturali:

Failure Case Tipico 1 (Falso Positivo borderline):

TRUE=Junk/Store (0) | PRED=Informative (1)

Testo: "Helldivers 2 Super Credits For Sale - IGGM | 1000 Super Credits \$2.52 Buy Now. Super Credits are the premium in-game currency that can be exchanged for a number of rare armor sets, weapons..."

Analisi: Il modello può etichettare uno store di microtransazioni come recensione utile. Questo accade perché il testo mescola astutamente termini tipici del gameplay ("in-game currency", "rare armor sets", "weapons") che normalmente caratterizzano un articolo giornalistico, confondendo la rete neurale nonostante la presenza di diciture commerciali ("Buy Now", "\$2.52").

Failure Case Tipico 2 (Falso Negativo borderline):

TRUE=Informative (1) | PRED=Junk/Store (0)

Testo: "Final Fantasy XV Review | Final Fantasy XV Review IGN 19800000 subscribers 17274 likes 2078653 views 28 Nov 2016 Square's storied RPG... <https://www.youtube.com/watch?v=...>"

Analisi: Il testo è la pura descrizione di una video-recensione legittima di IGN. Tuttavia, la presenza massiccia di numeri, metriche sociali slegate dalla lore ("subscribers", "likes", "views") e di un URL testuale grezzo ha storicamente indotto il modello a credere che si trattasse di una pagina spazzatura non informativa, spingendolo a scartare una fonte valida.

Ipotesi di Miglioramento (Future Work): Per arginare le incertezze su questi campioni ambigui, oltre che al più ovvio incremento dei campioni totali del dataset, si ipotizzano due strategie architetturali:

1. **Data Enrichment & Sintesi Ibrida:** Aumentare significativamente i campioni nel Training Set che presentano metadati grezzi di YouTube (likes, views) etichettati come "Classe 1", costringendo la rete ad associare quei termini al giornalismo video e non solo allo spam.

2. **Multiclassification:** Abbandonare la logica binaria in favore di una multiclassificazione (es. 0=Store, 1=Recensione, 2=Forum/Social/Video), applicando pesi differenti nella Loss Function per isolare le diverse semantiche testuali di Internet.

5.10 Deployment e Integrazione del Modello (Inferenza)

Al termine del processo di training, si è posta la necessità di integrare il modello validato all'interno della pipeline LangGraph (nel modulo `ml_manager.py`). Dal punto di vista dell'ingegneria del software, si è deciso di **non eseguire il merge persistente** dei pesi LoRA all'interno del modello base su disco. L'esportazione unificata avrebbe generato un artefatto monolitico di oltre 700 MB.

Al contrario, il sistema istanzia all'avvio una classe *Singleton* che applica il paradigma della **Dynamic Adapter Injection**:

1. Carica in memoria RAM il modello BERT base originale, mantenendolo nel suo stato pre-addestrato (frozen).
2. Inizializza l'oggetto `PeftModel`, il quale legge dalla directory di training unicamente la configurazione e i tensori addestrati di LoRA (`adapter_model.safetensors`, dal peso di circa 1 MB).
3. "Somma" dinamicamente in memoria questi piccoli adattatori sopra i layer di attenzione del Foundation Model.

Questo design privilegia la modularità rispetto alla velocità di inferenza pura: il modello non-merged è marginalmente più lento di una versione con pesi unificati, ma permette di sostituire o aggiornare le matrici LoRA senza ricaricare i pesi strutturali di BERT.

6 Valutazione del Sistema e Osservabilità

In conformità con le specifiche di progetto, l'architettura agentica è stata sottoposta a una rigorosa valutazione finale. L'analisi ha misurato la qualità dell'output generato, l'ancoraggio ai fatti (grounding), la resilienza agli errori e il costo computazionale, avvalendosi della piattaforma di osservabilità **LangSmith**.

6.1 Analisi Qualitativa dei Post e Grounding delle Fonti

La qualità testuale e l'affidabilità delle recensioni prodotte dal nodo *Writer* rappresentano il fulcro dell'intero ecosistema.

- **Analisi Qualitativa:** Sfruttando le capacità di instruction-following di Llama-3.3-70B, la stesura finale si discosta completamente dal classico tono enciclopedico ("GPT-like"). I prompt di sistema forzano l'agente a utilizzare uno stile da blogger, adottare un gergo videoludico assertivo e strutturare il post in un formato Markdown rigoroso (con sezioni separate per Lore, Gameplay, Grafica e Giudizio Finale). Le recensioni risultano così più leggibili, scorrevoli e coerenti con le aspettative del target di riferimento.
- **Assessment del Grounding e delle Fonti:** Per garantire l'assoluta assenza di allucinazioni, il sistema applica una politica di *Zero-Trust*. Il grounding è assicurato in due fasi:

1. A monte, il filtro **BERT+LoRA** elimina fisicamente lo spam e i siti e-commerce dai risultati di ricerca, proteggendo il RAG dall'inquinamento semantico.
2. A valle, il nodo *Summarizer* è costretto a compilare lo schema Pydantic `GameResearchExtraction`, che contiene il campo obbligatorio `fact_check_notes`. L'LLM deve annotare esplicitamente le eventuali discrepanze fattuali trovate tra le varie fonti web e riportare l'URL esatto di ogni citazione. Il *Writer* è vincolato a generare testo basandosi **esclusivamente** su questo payload purificato, annullando il rischio di invenzioni narrative.

6.2 Identificazione dei Failure Cases e Limiti Architettureali

L'unione tra l'elaborazione stocastica dell'LLM e i vincoli fisici dell'infrastruttura di inferenza espone il sistema ad alcune limitazioni e scenari di fallimento. Di seguito si analizzano i principali *Failure Cases* riscontrati e le motivazioni architettureali sottostanti:

1. **Failure Case 1: Devianze Semantiche e Disallineamento dai Prompt.** Sebbene i nodi logici (*Planner*, *Summarizer*, *Writer*) siano governati da rigidi schemi Pydantic e *System Prompt* precisi e severi, l'LLM presenta occasionali devianze comportamentali. Si sono verificati casi in cui il modello non compila gli schemi alla lettera (ad esempio, formattando la recensione finale omettendo alcuni header Markdown richiesti) o adotta un tono leggermente più enciclopedico del previsto.
Analisi: Queste fluttuazioni dimostrano la natura probabilistica degli LLM che, a volte, non riescono a seguire ciecamente istruzioni complesse stratificate in contesti molto lunghi. Tuttavia, queste leggere devianze non compromettono mai la solidità fattuale o l'esperienza utente complessiva del Blogger Autonomo.
2. **Failure Case 2: Troncatura Forzata del ReAct Loop (Limiti di Budgeting).** Il nodo *Researcher* opera tramite un ciclo ReAct potenzialmente infinito per scavare nei dettagli di un gioco. Per evitare di saturare i limiti di Rate (TPM/RPM) imposti dall'API e di generare costi insostenibili, l'architettura impone uno stop forzato (*Hard Limit*) dopo 5 iterazioni, a prescindere dal fatto che l'LLM abbia o meno raggiunto la soddisfazione informativa.
Analisi: Se il limite viene raggiunto, il lavoro di ricerca viene inevitabilmente "lasciato a metà", costringendo il *Writer* ad accontentarsi dei dati parziali raccolti fino a quel momento. Si tratta di un limite puramente infrastrutturale: con budget e risorse cloud scalabili, il tetto potrebbe essere innalzato, garantendo recensioni di profondità sempre maggiore.
3. **Failure Case 3: Saturazione delle API (Rate Limiting) e Latenza Globale.** L'ecosistema sviluppato è un sistema multi-agente estremamente proattivo: una singola esecuzione per generare un post può richiedere fino a 15-20 chiamate LLM distinte tra pianificazione, ricerca iterativa, riassunto e stesura. L'uso prolungato e intensivo del sistema porta rapidamente all'esaurimento della quota gratuita di API Key (es. Groq), causando il blocco temporaneo dei servizi (è spesso necessario, ad esempio, cambiare l'API KEY per continuare ad usare il sistema).
Analisi: Questo collo di bottiglia espone i limiti di latenza delle pipeline agentiche. Attualmente, l'orchestrazione completa richiede circa 3 minuti per la generazione di una recensione. In ottica *Enterprise*, l'ottimizzazione del numero di chiamate e l'uso di cluster dedicati (es. istanze private Llama-3) annullerebbero queste limitazioni hardware.

4. **Failure Case 4: Perdita di Risoluzione nella Named Entity Recognition (NER).** Il nodo *Memory Updater* è incaricato di leggere l'articolo approvato ed estrarre le entità (Topic, Boss, Meccaniche, Similarità) per aggiornare il Knowledge Graph.

Analisi: Si è riscontrato che l'LLM, operando su testi molto densi, si "perde" occasionalmente alcuni micro-dettagli, non riuscendo a estrarre il 100% delle informazioni presenti nel testo. Pur garantendo il salvataggio delle nozioni cardine, questo comportamento dimostra che i LLM generalisti non sono ancora estrattori NER perfetti, suggerendo la necessità futura di modelli *fine-tuned* esclusivamente per task di estrazione relazionale.

6.3 Parametri di Osservabilità in LangSmith

Per valutare oggettivamente le performance operative e i costi computazionali, l'intera pipeline è stata tracciata interfacciando LangGraph con la piattaforma **LangSmith**. L'analisi delle *Trace* ha evidenziato metriche vitali per il perfezionamento del sistema:

- **Tracciamento del Ragionamento (ReAct Loop):** LangSmith ha permesso di svelare la "scatola nera" del nodo *Researcher*. Si è potuto verificare visivamente l'aderenza dell'LLM al paradigma *Thought* $\rightarrow$ *Action* $\rightarrow$ *Observation*, confermando che l'agente invocava il `knowledge_graph_tool` come prima mossa assoluta (per inquadrare il contesto del gioco) per poi passare iterativamente agli strumenti web e all'approfondimento degli aspetti interessati dalla recensione tramite il RAG.
- **Consumo dei Token e Context Window:** Il monitoraggio ha rilevato picchi anomali di *Prompt Tokens* quando l'agente tentava di elaborare l'intero raw text dei siti web scaricati. Questa criticità di LangSmith ha guidato **la principale scelta ingegneristica del progetto**: lo sviluppo del tool `deep_read_article`. Invece di inviare documenti monolitici, si è data all'agente la facoltà di "paginare" gli articoli a blocchi tramite parametri di *Offset/Limit*, abbattendo drasticamente il consumo dei token e mitigando il fenomeno del *Lost in the Middle*.
- **Latenza e Tempi di Esecuzione:** La piattaforma ha dimostrato che la latenza inferenziale del modello è pressoché istantanea. Il vero *bottleneck* temporale è generato dai tempi di latenza fisiologici delle operazioni sul nodo **Researcher** (principalmente le chiamate API di Tavily).
- **Ispezione degli Input/Output dei Tool:** L'osservabilità dei payload JSON scambiati tra i nodi si è rivelata fondamentale per il collaudo del Knowledge Graph. L'ispezione dei metadati in uscita dal nodo *Memory Updater* ha permesso di correggere i prompt Pydantic che generavano entità duplicate, forzando l'LLM a estrarre esclusivamente i nomi canonici ("Soggetto — Relazione — Oggetto") per mantenere atomica la struttura relazionale di Neo4j. Ad esempio, il sistema ha imparato che "Final Fantasy 8" e "Final Fantasy VIII" sono la stessa entità, evitando di creare nodi separati per la stessa proprietà.

7 Conclusioni e Sviluppi Futuri

L’architettura realizzata dimostra la maturità tecnica delle pipeline Agentiche, a condizione che siano supportate da un’ingegnerizzazione del software meticolosa. L’integrazione strutturale di LangGraph, unita al grounding ontologico di Neo4j, al recupero ibrido di ChromaDB e ai **filtri neurali PEFT (BERT) addestrati su dominio specifico**, annulla la debolezza stocastica dei Large Language Models.

Per gli sviluppi futuri, oltre a quanto espresso in precedenza come possibili migliorie, l’analisi delle performance suggerisce il deployment di un LLM locale di dimensioni ridotte (es. Llama-3-8B) sottoposto a *Direct Preference Optimization (DPO)* per emulare nativamente il gergo videoludico, abbattendo ulteriormente i costi di inferenza e rendendo il Blogger Autonomo un ecosistema production-ready, affidabile e di livello industriale.